

Guía de Aprendizaje: Creación de un Vector Dinámico

Malak Sanchez

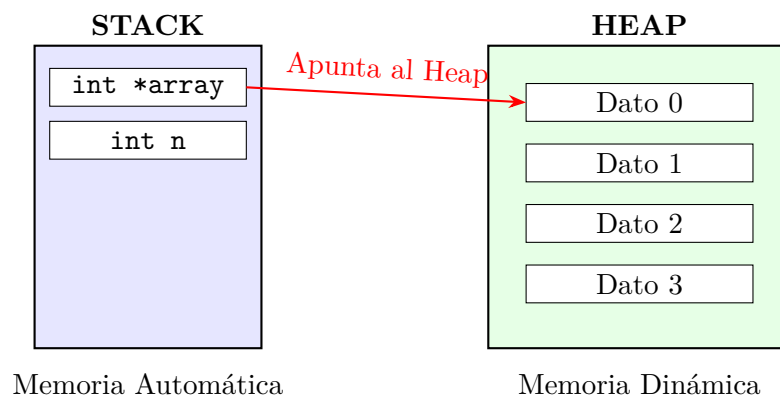
Workshop 2026-II

16 de agosto de 2026

1. Introducción: Stack vs Heap

Para entender la memoria dinámica, primero debemos diferenciar dónde vive cada variable en nuestro programa.

- **Stack (Pila):** Es memoria automática y rápida. Aquí viven las variables locales (como `int x`). Su tamaño se define al compilar. Cuando la función termina, esta memoria se libera automáticamente.
- **Heap (Montículo):** Es un gran bloque de memoria libre gestionado por el programador. Aquí vive la memoria dinámica. Es más lenta que el stack, pero su tamaño puede decidirse **mientras el programa corre**. Tú eres responsable de pedirla y liberarla.



2. Creación de un Vector Dinámico

Un **vector dinámico** es simplemente un arreglo cuyo tamaño se define en tiempo de ejecución.

2.1. El proceso en 3 pasos

1. **Declarar un puntero:** El puntero vivirá en el *Stack*.
2. **Pedir memoria:** Usamos `malloc` para reservar espacio en el *Heap*.
3. **Liberar memoria:** Usamos `free` cuando ya no necesitemos el arreglo.

3. Código de Ejemplo

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(){
5      int n;
6      printf("Enter vector size: ");
7      scanf("%d", &n);
8
9      // 1 and 2. Declare pointer and request memory in the Heap
10     int *array = (int *) malloc(n * sizeof(int));
11
12     // Check if allocation was successful
13     if(array == NULL){
14         printf("Error: Could not allocate memory.\n");
15         return 1;
16     }
17
18     // Use the array as a normal array
19     for(int i=0; i < n; i++){
20         array[i] = i * 10;
21         printf("array[%d] = %d\n", i, array[i]);
22     }
23
24     // 3. Free memory when finished
25     free(array);
26     array = NULL; // Good practice
27
28     return 0;
29 }
```

4. ¿Por qué usar malloc y sizeof?

La función `malloc` recibe la cantidad de **bytes** a reservar. Como no sabemos cuánto pesa un `int` en todos los sistemas, usamos `sizeof(int)`. Así, `n * sizeof(int)` garantiza espacio exacto para n enteros.

5. Resumen Visual

Cuando haces `int *v = malloc(3 * sizeof(int))`:

1. En el **Stack** se crea `v`.
2. En el **Heap** se buscan $3 \times 4 = 12$ bytes contiguos.
3. `v` guarda la dirección del primer byte de ese bloque en el Heap.